UF2405. UD3. Actividad 3_Ejercicio teórico-práctico "LENGUAJE ORIENTADOS A OBJETOS"

Datos de la actividad

Campo	Información
Unidad formativa	UF2405 — Modelo de programación web y bases de datos
Unidad didáctica	UD3 — Lenguajes de programación web
Actividad	Actividad 3 — Lenguajes orientados a objetos
Proyecto	actividad3-uf2405
Fecha	Octubre de 2026
Autor	[Nombre y apellidos]

1. Enunciado y alcance

La actividad solicita reproducir una página web de referencia mediante HTML y añadir interactividad con JavaScript para que, al pulsar un botón, aparezca el mensaje «Mensaje mostrado al pulsar el botón». La solución se organiza en archivos separados para la estructura, los estilos y el comportamiento.

2. Actividad 1: construcción de la página web

Se presenta la estructura principal de la página en index.jsp. El archivo combina HTML con enlaces a la hoja de estilos y al script externo.

2.1. Archivo index.jsp

```
<!DOCTYPE html>

<html lang="es">

<head>

  <meta charset="UTF-8">

  <meta name="viewport" content="width=device-width, initial-scale=1.0">

  <title>Título de la página de pruebas</title>

  <link rel="stylesheet" href="css/estilos.css">

</head>
```

```
<body>
  <div class="contenedor">
    <div class="contenedor-imagen">
      <img src="https://images.unsplash.com/photo-1506905925346-
21bda4d32df4?w=400"
        alt="Cielo con nubes y atardecer">
    </div>
    <hr>
    <p>
      <a href="https://www.ejemplo.com">Nombre del vínculo</a>
      es un vínculo a otro buen sitio web
    </p>
    <h1>Esto es un encabezado</h1>
    <h2>Este es un encabezado mediano</h2>
    <p>Envíen el correo a
      <a href="mailto:soporte@suempresa.com">soporte@suempresa.com</a>
    </p>
    <p>¡Este es un nuevo párrafo!</p>
    <p>¡Este es un nuevo párrafo!</p>
    <p class="texto-estilo">Esta es una nueva oración sin salto
      de párrafo, en cursiva negrita.</p>
    <hr>
    <button class="boton-mensaje" onclick="mostrarMensaje()">
      Pulsar para mostrar mensaje
    </button>
  </div>
  <script src="js/script.js"></script>
</body>
</html>
```

Nota de integración: aunque el archivo se denomina index.jsp, la estructura anterior es HTML válido dentro de una JSP. Si la entrega exige fecha y hora generadas en servidor, deberá añadirse la expresión JSP correspondiente en la posición deseada.

2.2. Archivo css/estilos.css

```
* {
    margin: 0;
    padding: 0;
    box-sizing: border-box;
}

body {
    background-color: #FFFF00;
    font-family: 'Times New Roman', Times, serif;
    padding: 20px;
    line-height: 1.6;
}

.contenedor-imagen {
    text-align: center;
    margin: 20px 0;
}

.contenedor-imagen img {
    width: 300px;
    height: auto;
    border: 1px solid #333;
}

hr {
    border: none;
    border-top: 2px solid #4a90e2;
    margin: 20px 0;
}
```

```
a {
  color: blue;
  text-decoration: underline;
}

h1 {
  font-size: 2.5em;
  margin: 20px 0 10px 0;
}

h2 {
  font-size: 1.8em;
  margin: 15px 0 10px 0;
}

p {
  margin: 10px 0;
  font-size: 1.1em;
}

.texto-estilo {
  font-style: italic;
  font-weight: bold;
}

.boton-mensaje {
  background-color: #4a90e2;
  color: white;
  border: none;
  padding: 12px 24px;
  font-size: 1em;
```

```
    cursor: pointer;
    border-radius: 5px;
    margin: 20px 0;
}

.boton-mensaje:hover {
    background-color: #357abd;
}
```

2.3. Justificación técnica de la actividad 1

El fondo amarillo se define mediante background-color: #FFFF00.

La imagen se centra con text-align: center en su contenedor.

Las líneas horizontales se personalizan con CSS.

Los enlaces se crean con la etiqueta <a>; el correo utiliza el protocolo mailto:.

Los encabezados h1 y h2 diferencian jerarquías de contenido.

Los párrafos utilizan <p> y la clase texto-estilo combina cursiva y negrita.

CSS se mantiene en un archivo externo, separado del HTML.

3. Actividad 2: interactividad con JavaScript

Se incorpora un botón cuyo evento onclick invoca la función mostrarMensaje(). La función utiliza alert() para mostrar el texto exacto solicitado.

3.1. Archivo js/script.js

```
/**
 * Muestra el mensaje solicitado al pulsar el botón.
 * Se ejecuta en el navegador del usuario.
 */
function mostrarMensaje() {
    alert("Mensaje mostrado al pulsar el botón");
    console.log("Botón pulsado. Mensaje mostrado al usuario.");
}

window.onload = function() {
    console.log("JavaScript cargado correctamente.");
```

```
  console.log("Fecha de carga: " + new Date().toLocaleString());
};
```

3.2. Integración del botón

```
<button class="boton-mensaje" onclick="mostrarMensaje()">
  Pulsar para mostrar mensaje
</button>

<script src="js/script.js"></script>
```

3.3. Justificación técnica de la actividad 2

El botón incluye un evento que llama a la función JavaScript.

mostrarMensaje() muestra una alerta con el texto exacto del enunciado.

El código se mantiene en js/script.js, separado de la estructura HTML.

Los console.log() permiten observar mensajes de diagnóstico desde las herramientas de desarrollo del navegador.

4. Objetivos y criterios de evaluación

4.1. C1: aplicar conceptos básicos del modelo de programación web

La solución integra HTML5 para estructurar la página, CSS3 para su presentación y JavaScript para la interacción del lado del cliente. El proyecto añade Java Servlet y JSP para el tratamiento de la petición y la preparación de la vista en el servidor.

4.2. CE1.8: características de lenguajes orientados a objetos

La parte Java del proyecto permite explicar las características de la programación orientada a objetos aplicadas a la recepción de solicitudes y a la preparación de la capa de presentación.

Característica	Aplicación en el proyecto
Herencia	Actividad3Servlet extiende HttpServlet y hereda su funcionalidad.
Polimorfismo	El Servlet sobrescribe doGet() y doPost() para definir el tratamiento de las peticiones.
Encapsulamiento	Los métodos de tratamiento se declaran protected conforme al contrato de la API.
Abstracción	La API de Servlet oculta parte de la complejidad de la recepción y gestión de peticiones HTTP.

4.3. Generación dinámica con JSP

JSP permite generar contenido en el servidor e integrarlo en la vista. Por ejemplo, la fecha puede calcularse y mostrarse mediante una expresión JSP:

<% String fecha = new java.util.Date().toString(); %>\n Fecha: <%= fecha %>

4.4. Separación de responsabilidades

Capa o recurso	Tecnología	Responsabilidad
Estructura	HTML5/JSP	Organización y contenido de la página.
Presentación	CSS3	Diseño visual y formato.
Comportamiento cliente	JavaScript	Respuesta a eventos e interacción.
Control	Java Servlet	Recepción y gestión de peticiones HTTP.
Vista dinámica	JSP	Generación de contenido en servidor.

5. Conclusiones

La propuesta responde a las dos tareas descritas: reproduce la página solicitada mediante HTML y CSS, e incorpora una acción JavaScript que presenta el mensaje requerido. Además, el proyecto permite documentar la relación entre Servlet y JSP y explicar conceptos básicos de orientación a objetos.

La valoración final debe basarse en la ejecución real del proyecto y en la comprobación de sus archivos. En particular, conviene verificar que la ruta de bienvenida, el mapeo del Servlet, las rutas CSS/JS y la fecha de servidor coinciden con la versión entregada.

6. Evidencias de funcionamiento

Inserte aquí capturas de pantalla obtenidas durante la ejecución real. No deben sustituirse por imágenes simuladas.

Captura 1. Página principal cargada: fondo amarillo, imagen, encabezados, párrafos y botón.

Captura 2. Alerta que aparece al pulsar el botón: «Mensaje mostrado al pulsar el botón».

Captura 3. Consola del navegador con los mensajes de diagnóstico.

Captura 4. Estructura de archivos del proyecto: Servlet, JSP, CSS, JavaScript y documentación.

7. Referencias

UF2405 — Modelo de programación web y bases de datos; certificado de profesionalidad IFCD0112.

Java Servlet Specification: https://javaee.github.io/servlet-spec/

HTML Living Standard: https://html.spec.whatwg.org/

MDN Web Docs — JavaScript: https://developer.mozilla.org/

Apache Tomcat: https://tomcat.apache.org/

Apache Maven: https://maven.apache.org/